Engineering-Konsole: Betrieb, Freigabe und Analyse

Stand 10. November 2026 · ursprünglich von Artan (26.10.2026), seither nachgeführt

Diese Anleitung beschreibt die Engineering-Konsole (`src/crime/platform_admin`), so wie sie im Repository liegt. Die Infrastruktur bleibt Grundlage: SeaweedFS/S3, die bestehenden Pipelines, Elasticsearch, Kibana, Jupyter und das vorhandene Monitoring. Was sich gegenüber Artans Lieferung geändert hat und warum, steht in `AENDERUNGEN.md` unter den Einträgen vom 30.10.2026.

`handbuch/ENGINEERING_KONSOLE.pdf` ist dieselbe Anleitung als PDF, erzeugt aus dieser Datei mit `python tools/build_konsole_pdf.py`. Von Hand geändert wird nur das Markdown.

1. Was sich durch die Integration ändert

Die Konsole steuert Änderungen und Freigaben. Ein einzelner Worker führt Betriebsimporte, isolierte Kandidatenbauten und die Analysebereitstellung nacheinander aus. Webserver und Worker sind getrennte Prozesse. Die Weboberfläche führt keinen ETL-Lauf selbst aus.

Ein aktiver Datenrelease ist ein unveränderliches Verzeichnis mit Manifest und SHA-256-Prüfsummen. Seine Kennung ist ein Name, nicht selbst die Prüfsumme. AKTUELL.json verweist auf den aktivierten Stand. Der bisherige Stand bleibt aktiv, solange ein neuer Kandidat nur berechnet, geprüft oder auf einen Merge wartet.

Die Konsole unterscheidet jetzt ausdrücklich:

- **Aktiver Release:** freigegebener Datenstand; bei der Erstübernahme gegebenenfalls dokumentierte Alt-Baseline.
- **Analyse-Bereitstellung:** Zustand des Exports dieses Standes nach Elasticsearch/Kibana und der abschliessenden Prüfung.
- **Zuletzt vollständig geprüft:** letzter Stand, dessen gesamte Analysebereitstellung erfolgreich beendet wurde.

Zwischen Datenfreigabe und fertig aktualisierten Analysediensten kann Zeit liegen. Elasticsearch wechselt seine Tabellen-Aliase einzeln. Ein Abbruch kann deshalb einen Teilstand hinterlassen; PARTIAL kennzeichnet ihn. Es gibt keine verteilte Transaktion über alle Dienste.

2. Die Funktionen und ihr Zweck

Aktiver Release und Fortschrittsanzeige

Die Anzeige beantwortet: Auf welchen Stand darf sich eine Analyse beziehen, und wo steht ein Auftrag? Ein Rebuild durchläuft Bronze-Snapshot, Silver einschliesslich Quellhistorisierung, Gold, Serving, Quality Gate, Impact und Freigabe/Release. Angezeigt werden nachgewiesene Schritte, keine erfundene Prozentzahl oder Restlaufzeit. Mehrere Vintages werden innerhalb derselben Stufe bearbeitet. SUCCEEDED eines Imports bedeutet nur, dass die Quelle geladen wurde.

Beispiel: Detroit Full ist erfolgreich, der Rebuild wartet auf Review. Bronze ist geladen; der aktive Release bleibt der vorherige. Eine grüne Importzeile ist keine fachliche Freigabe.

Mapping vorschlagen

Ein Engineer wählt Stadt, Quellcode und NIBRS-Zielcode, nennt eine Begründung und reicht den Vorschlag ein. Die Konsole speichert eine neue Revision mit Bezug auf die aktuelle Basis und berechnet einen Kandidaten. Führende Nullen in Quellcodes bleiben erhalten.

Die Korrektur steht neben dem Crosswalk, nicht darin. `config/iucr_nibrs_mapping.csv` und `config/micr_nibrs_mapping.csv` sind aus amtlichen Quellen erzeugt und werden gegen ihre Werkzeuge gehalten; die Konsole ändert sie nie. Ein Vorschlag wird eine Zeile in `config/mapping_overrides.csv` (Stadt, Quellcode, Zielcode, Begründung), und Silver wendet sie nach dem Crosswalk an. Eine solche Zeile trägt die Herkunft `admin_review` und wird als `ambiguous` eingeordnet: geprüft und entschieden, aber kein amtlicher Zuordnungsnachweis. Wer den Crosswalk-Wert wieder wählt, entfernt die Korrektur.

Konfiguration und Zeitpläne

Die JSON-Konfiguration ist die gemeinsame Quelle für Zeitraum, Qualitätsgrenze und Jobs. Die Quellimport-Formulare bearbeiten dieselbe Konfiguration. Sie stellen keine zweite unabhängige Konfiguration dar. Änderungen durchlaufen ebenfalls Kandidat und Review.

scope*start ist inklusive, scope*end_exclusive exklusiv. Der Zeitraum bestimmt die Jahre des verwalteten Gesamtimports und die Census-Vintages des Rebuilds. Er filtert nicht nachträglich beliebige bereits geladene Bronze-Jahre weg: Daten ausserhalb des festgelegten Zeitraums werden im Quality Gate beanstandet.

Der Wochenrhythmus des Datenlaufs (erster Montag Delta, zweiter Voll, dritter und vierter Delta, fünfter frei) kommt nicht aus diesen Zeitplänen, sondern aus `docker/srv01-konsole-datenlauf.sh`: Es reiht montags einen Gesamtimport ein. Die Zeitpläne der Konsole sind zusätzlich da und vorgabemässig alle aus; jeder Takt des Workers prüft sie mit.

Quellimporte und Ladelogik

Chicago Full lädt einen neuen Vollsnapshot für ein Tatjahr. Chicago Delta verwendet updated_on, das Wasserzeichen und den im Pipeline-Code festgelegten Lookback von 14 Tagen. Ohne Wasserzeichen entsteht zunächst ein Vollsnapshot. Der aktuelle Chicago-CLI besitzt keinen frei übergebbaren --lookback-days-Schalter.

Detroit Full lädt die angegebenen Jahre vollständig. Detroit Delta arbeitet je Jahr mit updated_at, Wasserzeichen und Pipeline-Lookback. Der ältere Daily-Modus verwendet dagegen ein Kalenderfenster bis zum abgeschlossenen Vortag. Er bleibt zur Kompatibilität vorhanden und ist als Legacy bezeichnet. Für den regulären Betrieb ist Delta vorgesehen.

Die Stadtbuttons öffnen die lesende Codeansicht. Full und Delta markieren relevante unterschiedliche Codebereiche. Gemeinsame Funktionen werden weiterhin vollständig angezeigt. Suche und Dateiauswahl ändern keinen Code. Die Vorschau zeigt den Code des Webserver-Abbilds. Historischer Worker-Code wird ohne Nachweis nicht behauptet.

Quelllauf-Nachweise enthalten Manifest, Run-ID, Parameter und Zeitstempel, soweit sie im tatsächlichen Manifest stehen. Job- und Impact-Verknüpfungen beruhen auf archivierten Manifest-Hashes. Ein fehlender Verweis bedeutet fehlenden Nachweis. Auch remote gespeicherte Bronze-Manifeste werden in der Konsole gelesen.

Review, Anträge und Konflikte

Die Review-Ansichten zeigen eigene Anträge, offene Reviews und Zuweisungen. Ein Reviewer kann einen Antrag übernehmen, lesen und fachlich entscheiden. Die Funktion ist eine interne Arbeitsliste, kein E-Mail-Postfach. Sie verschickt keine automatische E-Mail. Der Impact steht in der Konsole und als herunterladbares Archiv zur Verfügung.

Konkurrierende Vorschläge auf derselben Mappingzeile oder demselben Konfigurationsfeld werden angezeigt. Veraltete Vorschläge müssen auf die neue Basis übertragen und neu berechnet werden. Die Vier-Augen-Regel verbietet die Freigabe durch den Antragsteller. Eine Freigabe bindet den konkreten Impact-Lauf und seinen Berichtshash.

Jobs und Prüfprotokolle

Quellimporte laden Raw/Bronze. rebuild erstellt einen isolierten Vorschlag. replay verlangt denselben logischen Inhalt wie der aktive Release. quality prüft den verfügbaren Datenstand. Logs enthalten den Pipeline-Schritt und bei erkannten Fehlern die fehlende Abhängigkeit und Abhilfe. Ein Job im Status QUEUED benötigt einen laufenden Worker mit demselben absoluten Datenverzeichnis.

Geplante Gesamtimporte erscheinen unter Betriebsaufträge als batch. Ihr Log liegt unter `data/admin/jobs/<Auftrags-ID>/run.log`. Nach erfolgreichen Quellenimporten wird automatisch ein Rebuild eingereiht.

Ein geplanter Rebuild wird vom Qualitätstor freigegeben, nicht von einer Person (Entscheidung vom 30.10.2026). Der Cron-Datenlauf veröffentlichte jede Woche ohne Review; die Konsole behält das bei, mit dem Qualitätstor davor. Die Freigabe läuft über dieselben Prüfungen wie jede andere und steht unter dem Bearbeiter `quality-gate (automatic)`. Besteht der Kandidat das Tor nicht, wartet er auf eine Person. Alles, was eine Person vorschlägt oder anstösst, braucht weiter vier Augen.

Git und Betrieb

Git-Aktionen stehen direkt im geöffneten Antrag unter **Veröffentlichung**. Die Ansicht bündelt Git-Ampel, Branch-/Commit-Nachweise, die zum Zustand passende Aktion und das antragsbezogene Git-Protokoll. Eine zusätzliche zentrale Revisionsauswahl entfällt. **Commit / Branch / PR erstellen** verlangt die Engineer-Rolle, **Geprüften PR mergen** einen Release-Manager als zweite Person. Die serverseitigen Freigabe- und CI-Prüfungen bleiben unverändert.

Unter **Betrieb** trennen die Ansichten **Aufträge**, **Analyse-Bereitstellung** und **Sicherung** Verarbeitung, Bereitstellung und Wartung. **Aktiven Release erneut bereitstellen** liegt im zugeklappten Wartungsbereich und verwendet nur den aktiven, freigegebenen Release; es wird kein neuer Kandidat berechnet. Grün wird die Bereitschaft nur bei erfolgreicher Bereitstellung genau dieses Releases angezeigt. Unter **Sicherung** stehen der letzte Erfolgsnachweis, Speicherart und Archivschlüssel. **Konsolenzustand sichern** ist eine manuelle Operator-Aktion mit Begründung. Diese Sicherung ersetzt weder eine vollständige Quelldatensicherung noch ein Serverbackup; die Wiederherstellung bleibt ein kontrollierter Offline-Vorgang. Bedienung und Prüfgrenzen stehen ausführlich in `konsole-vorschlagseditor.md`.

Im gekoppelten Modus gilt: Impact archivieren, fachlich freigeben, PR/CI prüfen, exakten Merge nachweisen, Datenrelease aktivieren. Mapping- und Konfigurationsänderungen benötigen diesen Git-Schritt. Reine neue Quelldaten benötigen unter unveränderten Regeln keinen zusätzlichen PR, aber weiterhin einen geprüften Datenkandidaten. Auch dessen Build-Code wird im gekoppelten Betrieb gegen Main geprüft.

Die Betriebsaktionen umfassen PR-Erstellung, Merge-Anforderung, Rücknahme, erneute Archivierung, Backup und Analysebereitstellung. Der Worker führt sie aus; nur er hält das GitHub-Token. Die Konsole umgeht keine fehlgeschlagenen CI-Prüfungen.

`ADMIN_GIT_SCHUTZ` **entscheidet, was vor einem Merge geprüft wird.**

Wert	Merge nur, wenn	Wann
`produktiv` (Vorgabe, auch ohne Variable)	`main` die fünf GitHub-Schutzregeln trägt: strikte Statusprüfung mit mindestens einem Check, eine Freigabe, veraltete Freigaben verwerfen, Admins eingeschlossen	im produktiven Betrieb
`entwicklung`	jeder GitHub-Actions-Lauf auf dem exakten PR-Kopf abgeschlossen und nicht gescheitert ist, mindestens einer erfolgreich	solange `main` ohne Schutzregeln bleibt, damit direkt gepusht werden kann

Ein anderer Wert verhindert den Start. Der Modus steht im Nachweis unter `config/admin_reviews/`, im PR-Text und, wenn er `entwicklung` ist, als Banner in der Oberfläche.

Wechsel auf `produktiv`, wenn die Entwicklung abgeschlossen ist (bis dahin gilt `entwicklung`, auch auf srv01 — Entscheidung vom 30.10.2026): zuerst auf GitHub unter *Settings* → *Branches* für `main` die fünf Regeln setzen, dann in `docker-compose.srv01.yml` bei `admin` und `worker` `ADMIN_GIT_SCHUTZ: produktiv` eintragen. Ab dann geht jede Änderung auf `main` über einen Pull Request mit Freigabe, auch für die Entwickler, und ein Konsolen-PR braucht die Freigabe einer anderen Person als der Inhaberin des Tokens.

3. Was der Impact-Bericht tatsächlich misst

Der Bericht vergleicht den aktiven Release mit dem Kandidaten: hinzugefügte, entfernte und geänderte Datensätze, Delikte und Fälle, Wechsel von NIBRS-Code und Kategorie sowie Qualitätskennzahlen. Die Zählweise bleibt diejenige der Pipeline: Delikte entsprechen zählenden Gold-Zeilen; Fälle entsprechen counts_case-Repräsentanten. Fallzahlen sind nicht pauschal die Summe beliebiger Kategorien.

Vergleiche sind nach Stadt, Zeitraum und Deliktkategorie aufgeschlüsselt. Die vollständigen Detaildaten liegen als Parquet vor; die Anzeige im Browser ist begrenzt. Zusätzlich werden logische Fingerprints der Gold-/Serving-Tabellen verglichen. Damit werden auch Dimensions- und Nenneränderungen sichtbar. Nicht jedes geänderte Feld erhält automatisch eine eigene fachliche Visualisierung.

Beispiel mit **ausschliesslich fiktiven Zahlen**, keine Mapping-Empfehlung: Ein Detroit-Mapping verschiebt 120 zählende Zeilen aus einer Auffangkategorie in Betrug.

Messgrösse	Vorher	Nachher	Änderung
Detroit, Delikte gesamt	10 000	10 000	0
Auffangkategorie	500	380	-120
Betrug	800	920	+120
Unveränderte Datensätze	9 880	9 880	0
Datensätze mit geändertem Mapping	0	120	+120

Es entsteht kein zusätzliches Delikt. Die Verteilung ändert sich. Auswirkungen auf Fälle, Zeitaggregate, Raten und Qualitätsstatus werden aus den wirklichen Daten berechnet; sie dürfen nicht aus der Beispielverschiebung geraten werden. Ein Erstaufbau ohne Baseline und ein alter Release ohne Silver-Nachweise werden als eingeschränkt vergleichbar gekennzeichnet.

Pro Änderungsantrag und Berechnungsversuch bleiben Bericht, Detailartefakte, Hashes und Freigaben archiviert. Ein fehlgeschlagenes Archivieren verhindert die Freigabe; nach Behebung kann archive-retry verwendet werden.

4. Lokale Tests unter Windows: chronologische Einrichtung

Voraussetzung: Python 3.12 oder 3.13, entpacktes Projekt und freie Platte für Quellen, Snapshot, Kandidat und Release. Für vollständige Quellenimporte werden Netzwerkzugriff und gegebenenfalls API-Schlüssel benötigt. Die Server-Lockdatei requirements.lock ist auf Python 3.13 ausgelegt. Für einen lokalen Einstieg werden die Paketbereiche installiert.

Schritt 1: Projekt und Umgebung vorbereiten

Im Projektordner, in PowerShell:

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[admin,dev]"
$ProjectRoot = (Get-Location).Path
$DataRoot = Join-Path $ProjectRoot "data-konsole"
$env:ADMIN_RELEASE_MODE = "separate"
$env:ADMIN_OBJECT_STORE = "local"
$env:STORAGE_BACKEND = "local"
$env:PYTHON_DOTENV_DISABLED = "1"
$env:DUCKDB_MEMORY_LIMIT = "896MB"
$env:DUCKDB_THREADS = "2"
$env:DUCKDB_TEMP_DIRECTORY = Join-Path $DataRoot "duckdb-tmp"
```

Korrekt ist -e als zusammenhängende Option; nicht `- e`. Die direkte Ausführung des venv-Python benötigt keine Activate.ps1 und keine Änderung der PowerShell-Ausführungsrichtlinie. Die Speichergrenze ist eine Startkonfiguration, keine Zusage, dass jeder Full Load damit erfolgreich ist.

Schritt 2: Zeitraum festlegen, dann initialisieren

Für die Mehrjahresstrecke 2021 bis 2026 kann das Beispiel verwendet werden. Der Datenordner aus Schritt 1 ist **bewusst** `data-konsole`, nicht `data`: `enable-managed` sperrt danach `run_all` und alle

Pipelines für dieses Verzeichnis, und der gewohnte Arbeitsbestand wäre für die normale Entwicklung blockiert.

```
Copy-Item .\docs\console-settings.example.json `
  .\config\platform_admin_settings.json
.\.venv\Scripts\python.exe -m platform_admin init `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --operator Bootstrap
.\.venv\Scripts\python.exe -m platform_admin enable-managed `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --operator Bootstrap
```

Nur vor der **ersten** Initialisierung kopieren und den Zeitraum fachlich prüfen. init bewahrt vorhandene Revisionen. Bei einer bestehenden Konsole muss eine Konfigurationsänderung regulär vorgeschlagen und freigegeben werden. Ohne Bootstrap-Datei verwendet init den bisherigen lokalen Testzeitraum 2024. enable-managed dokumentiert die Übernahme und sperrt direkte schreibende Pipeline-Einstiege in diesem Datenverzeichnis. Ein vorhandener Release erhält dadurch keine erfundene rückwirkende Freigabe.

Schritt 3: Webserver starten, Terminal offen lassen

Im selben Terminal und mit den oben gesetzten Variablen:

```
.\.venv\Scripts\python.exe -m platform_admin serve `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --operator Engineer --port 8765
```

Die ausgegebene vollständige URL inklusive Sitzungstoken öffnen. Die Engineering-Konsole ist bereits die Flask-Webanwendung; es gibt lokal keinen dritten separat zu startenden Flask-Webserver. Dieser lokale Server bindet nur 127.0.0.1.

Schritt 4: Worker in einem zweiten Terminal starten

Im selben Projektordner zuerst die Variablen aus Schritt 1 erneut setzen; Umgebungsvariablen gelten pro Terminal. Dann:

```
.\.venv\Scripts\python.exe -m platform_admin worker `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --operator Worker
```

Terminal geöffnet lassen. Der Worker verarbeitet die Warteschlange. --once verarbeitet nur einen Tick und ist kein dauerhaft laufender Worker.

Schritt 5: Quellen laden und Rebuild prüfen

Ein Gesamtimport lädt auch die notwendigen Census- und Wetterquellen. In einem weiteren Terminal mit denselben Variablen:

```
.\.venv\Scripts\python.exe -m platform_admin.batch `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --mode full --operator Engineer --slot local-initial-001
```

Der Batch verwendet die genehmigte Konfiguration. Ein erneuter Aufruf mit demselben slot erzeugt keinen Doppelauftrag, auch nach einem Fehler. Für einen bewussten neuen Versuch einen neuen slot verwenden. Einzelne Chicago-/Detroit-Buttons laden nur die jeweilige Kriminalquelle. Für einen vollständigen Rebuild reichen diese beiden Quellen allein nicht aus.

Schritt 6: Zweite Review-Sitzung

```
.\.venv\Scripts\python.exe -m platform_admin serve `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --operator Reviewer --port 8766
```

Die zweite URL öffnen, Antrag und Impact prüfen und freigeben. Der lokale Operatorname ist eine Testidentität, kein authentifizierter Benutzer. Im Serverbetrieb ersetzt Cloudflare Access diese Namensangabe.

Für lokale Tests ohne Elasticsearch/Kibana bleibt ADMIN*AUTO*SERVING ungesetzt. Der freigegebene Release ist dann verfügbar; ein erfolgreicher extern bereitgestellter Analysebestand wird nicht behauptet. Sind die lokalen Analysedienste gestartet, ihre regulären ELASTICSEARCH*URL und KIBANA*URL setzen und unter Betrieb → Analyse-Bereitstellung → Wartung die Aktion „Aktiven Release erneut bereitstellen“ einreihen. Bei SERVING_BETRIEB=server verwendet die bestehende Zielarchitektur die Bereitstellung ohne Elasticsearch-/Kibana-Ladeschritte.

5. Betrieb auf srv01

Die Konsole läuft auf srv01 als zwei Dienste aus `docker-compose.srv01.yml`, beide aus **demselben Abbild** wie die Pipeline (Flask, PyJWT und gunicorn stehen in `requirements.lock`):

Dienst	Was	Speicher	Erreichbar
`admin`	Webdienst, dauernd	160 MiB	`urbancrime-admin.datenwerft.ch` über Traefik, hinter derselben Access-Anwendung wie alles unter `urbancrime-*`
`worker`	Stapellauf: ein Takt, höchstens ein Auftrag, dann Ende	1'280 MiB	gar nicht

Der Worker läuft **nicht dauernd**: Er führt die Pipelines aus und bräuchte dauernd deren 1'280 MiB, rund 1,3 GiB mehr als srv01 im Dauerbetrieb frei hat (FR-13). `docker/srv01-konsole-takt.sh` startet ihn alle fünf Minuten über dieselbe Sperre wie der Datenlauf, aber ohne zu warten — läuft gerade etwas, entfällt der Takt. Aufträge aus der Oberfläche beginnen deshalb mit bis zu fünf Minuten Verzug.

Der Kontrollzustand — SQLite mit Revisionen, Review-Entscheidungen und Git-Nachweisen — liegt im eigenen Datenträger `urbancrime_konsole`. Er lässt sich nicht nachrechnen und gehört in die Serversicherung.

Einrichten

Geführt in `offene-punkte.md` als S-5 bis S-10, in dieser Reihenfolge:

1. **Tunnel-Route** `urbancrime-admin.datenwerft.ch` auf `http://traefik:80` (S-5).
2. `.env` **des Stapels** (S-6): `CF_ACCESS_ISSUER` und `CF_ACCESS_AUD` aus der Access-Anwendung — ohne sie startet `admin` nicht —, `ADMIN_ROLE_BINDINGS` als JSON (stabile Access-`sub`-Kennung → Rollen; mindestens zwei Personen, sonst gibt es kein Vier-Augen-Prinzip), `ADMIN_METRICS_TOKEN` und `ADMIN_GITHUB_TOKEN` (Fine-grained, nur dieses Repository: Contents und Pull requests Lesen und Schreiben, Actions Lesen, ab `produktiv` zusätzlich Administration Lesen; nur der Worker liest es).
3. `urbancrime_konsole` **in** `backup.volumes` (S-7).
4. **In Betrieb nehmen** (S-8). Vorher `docs/console-settings.example.json` als `config/platform_admin_settings.json` ins Abbild bringen oder nach `init` als Einstellungsvorschlag einreichen — ohne sie setzt `init` den Zeitraum auf **2024 allein**, und jeder Gesamtimport lüde nur diesen Jahrgang.

```
dc() { docker compose --env-file .env -f docker-compose.srv01.yml "$@"; }
dc config --quiet
dc up -d admin
dc run --rm worker python -m platform_admin init \
  --project-root /app --data-root /app/data --operator christian
```

Danach in `deploy/cron/urbancrime.crontab` die Takt-Zeile einkommentieren und `crontab deploy/cron/urbancrime.crontab` neu laden.

5. `admin` **nachmessen**: `docker stats` nach einigen Minuten Betrieb. Am 31.10.2026 gemessen (S-9): 52 MiB in Ruhe, Spitze 66 MiB. Impact-Downloads werden gestreamt und heben den Speicher nicht.

Auf den verwalteten Betrieb umstellen

Bis hierher läuft der bisherige Cron-Datenlauf unverändert weiter, und die Konsole ist ein Werkzeug für Vorschläge. Verwaltet wird der Bestand erst mit `enable-managed` (S-10):

```
dc run --rm worker python -m platform_admin enable-managed \
  --project-root /app --data-root /app/data --operator christian
```

Im selben Zug in `deploy/cron/urbancrime.crontab` die Zeile für `srv01-datenlauf.sh` aus- und die für `srv01-konsole-datenlauf.sh` einkommentieren, dann den Crontab neu laden. Danach sperrt `storage.verwaltet` `run_all` und `release` für alles ausser dem Worker; der alte Datenlauf scheiterte ab dieser Minute. `enable-managed` holt im S3-Betrieb den aktiven Release lokal und dokumentiert ihn als Alt-Baseline, ohne ihm eine nachträgliche Freigabe zu erfinden.

Der Worker übernimmt dann alles, was der Datenlauf tat: Quellen laden, Kandidat bauen, Freigabe durch das Qualitätstor, Elasticsearch, Kibana, Startseite und `pruefe_stand`, danach Drift und Sicherungssatz, und nach jedem Takt mit Arbeit das Laufprotokoll `pipeline_run.json` und den Betriebsstand. Ein Wächter (`tests/platform_admin/test_datenlauf.py`) hält, dass jeder Schritt des Datenlaufs im Worker

vorkommt. Nicht übernehmen kann er `compose up` für Elasticsearch, Kibana und `start` — im Container gibt es kein Docker; die Dienste laufen `unless-stopped`.

Am ersten Montag nach der Umstellung im Betriebsblick prüfen, dass Protokoll, Drift und Sicherungssatz angekommen sind.

Ein neues Abbild ist weiter ein eigener Betriebsschritt (`docker/srv01-abbild-einsetzen.sh`); ein Merge aktualisiert keinen laufenden Container. Nie während eines laufenden Auftrags umstellen.

6. Analyse und Monitoring

Die bestehenden Elasticsearch-Sterntabellen, Geo-Indizes, Raten und Kibana-Dashboards werden durch die vorhandenen Lader erstellt. Die alte vereinfachte Konsolen-Indexstruktur wird nicht mehr verwendet. ClickHouse ist in diesem Zielstand keine Pflicht und wird von diesem Adapter nicht geladen.

Notebooks 34 und 37: Datenstand und Analysesitzung (E-5)

Die Notebooks lesen einen ausdrücklich gewählten, vollständig aufgebauten Pipeline-Bestand. Ein Ergebnis-Release enthält nicht alle dafür benötigten Bronze- und Silver-Daten. Notebook 34 vergleicht zusammengehöriges Silver und Gold; Notebook 37 untersucht `silver/source_change` und Bronze-Manifeste. Sie werden deshalb nicht pauschal auf das aktive Release umgeleitet.

Nach einem erfolgreichen lokalen Gesamtlauf liegt der Herkunftsnachweis unter `<Datenwurzel>/.analyse/stand.json`. Im verwalteten Betrieb entsteht er nach technisch erfolgreichem Kandidatenbau unter `data/admin/jobs/<Job-ID>/data/`. Er benennt Aufbaulauf beziehungsweise Job-ID, Code-Stand, gebaute Schichten, Datei-Prüfsummen und gegebenenfalls die zugehörige Release-ID. Die Kandidatenkennung ist **keine Review-Freigabe**. Welcher Job zum aktiven Release gehört, hält der **Job-Zeiger** `jobs/ANALYSE.json` fest: Beim Aktivieren eines Release trägt `finish_publication` dessen Job ein. Die Startzelle liest das Release aus `releases/AKTUELL.json` und schlägt es im Zeiger nach, nicht den jüngsten Eintrag. Schaltet jemand auf ein älteres Release zurück, liest sie dessen Daten. Fehlt für ein Konsolen-Release (`approved-…`) der Eintrag oder sein Ordner, bricht sie ab, statt auf einen anderen Stand auszuweichen. Für jedes andere aktive Release, etwa das des Cron-Datenlaufs, bleibt die Datenwurzel selbst. Das jeweils letzte `pipeline_run.json` ersetzt diesen Nachweis nicht.

Die Startzelle prüft die benötigten Dateien samt Prüfsummen und zeigt ihre Herkunft. Fehlende, veränderte oder unvollständig aufgebaute Daten führen zum Abbruch. Eine leere, erfolgreich gebaute `source_change`-Tabelle ist zulässig, belegt aber keine tatsächlichen Quelländerungen.

Die Analysesitzung hält eine exklusive Datensperre bis zur Abschlusszelle `analyse.beenden()` oder zum Ende des Kernels. Der Worker und die vorgesehenen schreibenden Einstiege nutzen dieselbe Sperre je Datenwurzel. Bei belegtem Bestand bleibt Worker-Arbeit eingereiht. Nach einem Zellfehler die Abschlusszelle manuell ausführen oder den Kernel herunterfahren; ein geschlossener Browser-Tab allein beendet den Kernel nicht. Nach Sitzungsende vor weiteren Abfragen die Startzellen erneut ausführen.

Vorbereitung auf srv01:

1. Neue Pipeline- und Jupyter-Images übernehmen. Die Compose-Datei bindet `datensperren` für beide Dienste und die Pipeline unter `/app/datensperren` ein und setzt `DATA_LOCK_ROOT` darauf. Gleiche Datenwurzeln müssen in den Containern denselben absoluten Pfad tragen.

2. Einen erfolgreichen Konsolen-Aufbau mit vollständigen Job-Daten bereitstellen. `jobs/` muss im `konsole`-Volume existieren. Jupyter erhält ausschliesslich diesen Unterordner als nur lesenden Subpath; die Kontrolldatenbank und Worker-Tokens im Elternordner bleiben ausserhalb dieses Mounts.
3. Den leeren Mountpunkt `admin/jobs` einmalig im `arbeitsdaten`-Volume vorbereiten, bevor Jupyter das Volume nur lesend einhängt:

```
dc run --rm --no-deps --entrypoint python pipeline -c "from pathlib import Path;
Path('/app/data/admin/jobs').mkdir(parents=True, exist_ok=True)"
```

4. `CRIME_ANALYSIS_ROOT` auf `/app/data` lassen: Die Startzelle findet den Job des aktiven Release über den Job-Zeiger. Ein fester Job-Ordner in der `.env` geht weiter, umgeht dann aber den Zeiger. In jedem Fall wird ein passender Herkunftsnachweis verlangt. Es werden weder automatisch Daten von S3 geholt noch andere Datenstände als Ersatz verwendet.
5. Für die Abnahme ausdrücklich die neuen Vorlagen unter `/vorlagen` in einen separaten Arbeitsordner unter `/app/arbeit` kopieren. Ein Image-Wechsel überschreibt bestehende Notebook-Arbeitskopien nicht.

Ein lokaler Aufbau bleibt mit `STORAGE_BACKEND=local` möglich. Für dessen Notebooks kann `CRIME_ANALYSIS_ROOT` vor dem Kernelstart auf einen anderen vollständigen lokalen Bestand zeigen. Details und Befehle stehen in `notebook-analysesitzungen.md`.

Nachweisstand am 31.10.2026: Beide Notebooks liefen lokal unter Windows und anschliessend unter Linux/Docker vollständig durch. Gegenseitige Leser-/Worker-Sperren, Freigabe und Volume-Schreibschutz wurden mit isolierten Test-Volumes bestätigt. Der dauerhafte Prüfnachweis nennt Datenstand, Images und Grenzen. Die Abnahme auf srv01 mit einem echten Konsolen-Job und die Auswertung tatsächlicher Quelländerungen bleiben offen; E-5 ist deshalb noch nicht als auf srv01 abgenommen markiert.

Der Betriebsblick bleibt derselbe. Der Worker schreibt nach jedem Takt mit Arbeit `pipeline_run.json` im Format, das `orchestration.run_all` für den Cron-Lauf schreibt, und rendert daraus `serving.betriebsstand` — dieselbe Metrikdatei, dasselbe Brett in Grafana. Ein leerer Takt überschreibt nichts. Die Konsole hat zusätzlich einen eigenen Endpunkt `/metrics` (Bearer `ADMIN_METRICS_TOKEN`); dass Prometheus ihn abholt, ist noch nicht eingerichtet (C-7).

7. Fehlerdiagnose und Wiederanlauf

QUEUED bleibt stehen

```
Get-CimInstance Win32_Process |
  Where-Object { $_.CommandLine -match "platform_admin (serve|worker)" } |
  Select-Object ProcessId, ParentProcessId, CommandLine |
  Format-List
Get-NetTCPConnection -LocalPort 8765 -State Listen |
  Select-Object LocalAddress, LocalPort, OwningProcess
```

Lokal muss mindestens ein Worker laufen; auf srv01 kommt alle fünf Minuten ein Takt — ein Auftrag, der länger als fünf Minuten in QUEUED steht, heisst dort, dass die Takt-Zeile im Crontab fehlt oder die Sperre

belegt ist (`logs/konsole-takt.log`). Webserver und Worker müssen denselben absoluten project-root und data-root nutzen. Zwei Python-Zeilen können unter Windows ein venv-Launcher und dessen Kindprozess sein; sie beweisen nicht automatisch zwei unabhängige Server. ParentProcessId und Portbesitzer prüfen. Nur die identifizierte überzählige Sitzung beenden, vorzugsweise mit Ctrl+C in ihrem Terminal.

RUNNING bleibt nach einem Abbruch stehen

Zuerst Worker und Pipeline-Kindprozesse prüfen und kontrolliert stoppen. Logs lesen. Ein KeyboardInterrupt bedeutet einen abgebrochenen Lauf, nicht einen noch rechnenden Import. Vor einer manuellen Statuskorrektur die Konsole ebenfalls stoppen und data/admin sichern. Bei hartem Prozessende kann worker.lock liegenbleiben; diese Sperre nur nach nachgewiesenem Ende **aller** Schreiber entfernen. Ein Container-PID ist nicht unbedingt derselbe wie der Host-PID.

```
# Nur nach bestaetigtem Stillstand aller Schreiber:
Copy-Item .\data\admin .\admin-recovery-backup -Recurse
# Nur falls die verwaiste Datei noch existiert:
Remove-Item .\data\admin\worker.lock
.\.venv\Scripts\python.exe -m platform_admin recover-job `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --job-id VOLLSTAENDIGE_JOB_ID --operator Operator `
  --reason "Prozesse beendet; Import nach Abbruch kontrolliert abgeschlossen"
```

recover-job schliesst nur RUNNING-Import-/Prüfjobs ab, keine Impacts. Für einen Impact gibt es abandon-impact mit --impact-id und Begründung. Für eine Betriebsaktion recover-operation mit --operation-id. Danach Worker neu starten und einen **neuen** Auftrag einreihen. Es wird kein halbfertiger Lauf als erfolgreich markiert.

Rebuild scheitert an fehlender Abhängigkeit

Beispiel: „Kein vollständiger Bronze-Lauf für tiger_tract, Vintage 2024, Staat 17“. Es fehlt eine Bronze-Quelle für Silver, nicht einfach eine bereits vorhandene Silver-/Gold-Datei. Ein Gesamtimport mit der konfigurierten Jahresspanne lädt Chicago, Detroit, TIGER Tracts, TIGER Places, ACS und Wetter. Fehlende API-Schlüssel zuerst beheben. Anschliessend neuen Rebuild ausführen. Im verwalteten Betrieb Quellen über Batch oder Konsole starten, nicht direkt am Worker vorbei.

Weitere Fälle: fehlende DuckDB-Spatial-/ICU-Erweiterungen, API 401/403, Rate Limit 429, Netzwerkfehler, fehlende Python-Pakete, Dateiberechtigungen und voller Datenträger. Das konkrete Job-Log enthält die erste relevante Ursache. Bei OOM DuckDB-Grenzen, freien Plattenplatz für Auslagerung und parallele Containerlast prüfen. Ein zusätzlicher Silver-Container vergrössert das verfügbare RAM nicht.

ARCHIVE*PENDING, AWAITING*GIT und PARTIAL

ARCHIVE*PENDING: S3-Zugang und Platz prüfen, danach Archivierung wiederholen. AWAITING*GIT: PR, CI, Branch-Schutz und Merge-Nachweis prüfen; nicht den Freigabemodus umschalten, um den konkreten Auftrag freizubekommen. PARTIAL: Bereitstellungslog unter `data/admin/serving/<Release-ID>/run.log` prüfen, Analysedienste reparieren und Bereitstellung des aktiven Releases erneut einreihen. Der Datenrelease bleibt dabei unverändert.

Aufräumen und Backup

Job-Daten. Jeder Kandidat liegt mit einer vollständigen Kopie seiner Eingaben unter `jobs/<Job-ID>/data/`, je einige hundert MB, und ging bisher jeden Tag mit in die Kontrollsicherung. Der Worker entfernt diesen Ordner in jedem Takt vor der Sicherung, sobald ihn niemand mehr braucht (`platform_admin.job_data.prune`). Die Nachweise daneben (`run.log`, Auswirkungsbericht, `result.json`) bleiben immer. Geschützt sind:

- eingereihte und laufende Jobs sowie Kandidaten, deren Freigabe oder Publikation noch läuft (`AWAITING_GIT`, `APPROVED_PENDING_PUBLICATION`, `ARCHIVE_PENDING`);
- Kandidaten im Review (`READY`, `BLOCKED`), solange ihre Basis das aktive Release ist; danach sind sie nicht mehr freigabefähig;
- der Job des aktiven Release und die zuletzt aktivierten davor, zusammen `ADMIN_JOB_DATA_KEEP` (Vorgabe 2), damit ein Zurückschalten seine Daten findet.

Liest gerade eine Analysesitzung einen Ordner, bleibt er bis zum nächsten Takt stehen. Entfernte Jobs verschwinden auch aus dem Job-Zeiger. Der Job des aktiven Release bleibt auch ohne Zeiger-Eintrag, etwa wenn er vor dem Job-Zeiger aktiviert wurde.

Was die Regel entfernen würde, zeigt ein Trockenlauf mit Grund und Grösse je Job; ohne `--dry-run` räumt derselbe Befehl sofort:

```
dc run --rm worker python -m platform_admin prune-job-data --dry-run --project-root /app
--data-root /app/data
```

Im verwalteten Modus bleibt der Aufräum-Trockenlauf lesend verfügbar. Destruktives Aufräumen ist zunächst gesperrt, weil die bisherigen Retentionsregeln offene Kandidaten und Prüfarchive nicht vollständig modellieren. Das ist eine bewusst verbleibende Betriebseinschränkung, keine Behauptung einer fertig integrierten Löschautomatik. Ein Full Load hebt andere Pin-/Referenz-/Historisierungsregeln nicht automatisch auf.

Die Backup-Aktion sichert SQLite konsistent sowie Kandidaten, Revisionen, Impact-Archive, Bereitstellungsnachweise und die lokalen Releases, **die der Eimer noch nicht hält**. Ein Release, dessen `release.json` mit denselben Bytes unter `releases/` im Eimer liegt, kommt nicht ins Archiv — `release.json` geht immer zuletzt hoch, das Release ist dann vollständig dort. `backup.json` nennt beide Listen (`releases_in_bucket`, `releases_archived`). Bis zum 10.11.2026 kamen alle Releases mit: jedes tägliche Archiv war auf 1,74 GB gewachsen, zwölf davon belegten 11,2 GB, der Eimer lief über sein Kontingent, wurde schreibgeschützt, und der Volllauf des Tages scheiterte daran. Im S3-Modus liegt das Archiv unter backups/control/; der Worker sichert täglich nach einem abgeschlossenen Tick. Lokal liegt es unter data/backups/control/ und trägt dort alle Releases, weil es keine zweite Kopie gibt. Nach jeder geprüften neuen Sicherung bleiben die neuesten `ADMIN_BACKUP_KEEP` (Vorgabe 7) Sicherungen; ältere werden samt allen Fassungen endgültig gelöscht, die gerade geschriebene nie. Andere Schlüssel unter dem Präfix bleiben unberührt. Das ist keine unabhängige Katastrophensicherung: Auf srv01 gehört der Datenträger `urbancrime_konsole` in die Serversicherung (S-7). Der kuratierte Sicherungssatz (`orchestration.sicherungssatz`) umfasst die Kontrolldatenbank nicht.

restore-backup verlangt ein leeres Ziel. Nach Restore denselben absoluten Datenpfad verwenden, da die Kontrolldatenbank Pfade enthält. Die Releases unter `releases_in_bucket` holt man danach aus dem Eimer (`python -m orchestration.release holen <Kennung> --data-root <Ziel>`). Zuerst Dienste stoppen, Integrität prüfen, ausstehende Operationen kontrollieren und erst dann den Worker starten. Alte

Auditdaten in der Kontrolldatenbank werden nicht automatisch gelöscht; nur ältere Sicherungsarchive fallen weg.

8. Was geprüft ist und was nicht

Die Tests der Konsole liegen unter `tests/platform_admin/` und laufen mit der ganzen Suite auf jedem Pull Request: echte SQLite- und Parquet-Dateien, die produktiven Pipeline-Einstiege auf kleinen Testdaten, kontrollierte Doppel für S3, GitHub und das Serving. Die Beschreibung auf srv01 halten `tests/test_srv01_compose.py` und `tests/test_srv01_konsole_takt.py`.

Nicht geprüft ist alles, was erst auf srv01 entsteht: der Start der Container, die Anmeldung über Cloudflare Access, ein echter GitHub-Merge, ein Export gegen die laufenden Elasticsearch- und Kibana-Dienste, der Speicher unter Last und eine Wiederherstellung aus der Sicherung. Das ist Teil der Einrichtung (S-5 bis S-10), nicht dieser Anleitung.